Module 09 — Jinja and Macros

Tier: 🟡 Working Effectively · **Duration:** 90 min · **Prerequisites:** Module 08

> **Why this module exists:** You've been using Jinja since Module 03 — `{{ ref() }}`, `{% if is_incremental() %}`, `{{ config() }}`. But you've been a consumer, not an author. Once a project grows past a handful of models, you'll notice the same structural patterns repeated: the same CAST expression in 12 staging models, the same surrogate key pattern in every Silver model. Macros solve this. This module shows you when that's worth doing, how to write a macro correctly, and — just as importantly — when to leave the logic in plain SQL instead.

In Module 08, you used `{{ var() }}` to substitute constants into SQL. That was Jinja in action. This module teaches you to write your own Jinja templates — macros — so you can define reusable SQL patterns that work across many models.

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
00:00	10 min	Recap Module 08	Confirm seeds and variables before advancing	Q&A	Answer from memory	—	Ask all 4 prep questions cold. Probe: "where does `dbt seed` load data?" and "what is the difference between a seed and a source?" These connect directly to understanding which layer macros operate on.	All 4 correct
00:10	10 min	Jinja syntax review — the three delimiters	Consolidate `{{ }}`, `{% %}`, `{# #}` with examples specific to macros	Present	Fill in delimiter table	This doc	This is review from Module 03 but focused on macro-authoring context. Deliberately show the broken macro using `{{ macro ... }}` to make the delimiter distinction concrete.	"Which delimiter do you use to define a macro?"
00:20	15 min	Built-in dbt variables + macro basics	Know what `{{ this }}`, `{{ target }}`, `{{ model }}` compile to; understand what a macro is	Present	Annotate compiled outputs	This doc	Show `dbt compile` output for each variable live. Key point: macros are just parameterised Jinja templates — they compile down to SQL.	"What does `{{ this.schema }}` return in your dev environment?"
00:35	15 min	Live demo — write the safe_cast macro	See a macro written from scratch, compiled, and used in a model	Live demo	Follow along in editor	Demo script below	Write the macro file, call it in a model, run `dbt compile`, show the compiled SQL. Do not skip the compile step — that's the payoff.	"What SQL does `{{ safe_cast('amount', 'DOUBLE') }}` compile to?"

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
00:50	10 min	When to use macros vs. CTEs vs. ephemeral	Know the decision criteria so macros aren't overused	Present	Decision table	This doc	This is the hard part. The antipattern — 50-line Jinja macros encoding business logic — is common and genuinely harmful. Be direct: if the macro needs comments to be readable, the logic belongs in SQL.	"Name one thing that should be a macro and one thing that should never be a macro"
01:00	5 min	Hooks and packages	Know `pre-hook` / `post-hook` syntax and `packages.yml` + `dbt deps`	Present	Read YAML snippet	This doc	Brief overview only. Hooks are rarely written from scratch — usually added for constraint DDL or audit logging. The key thing to know is that they exist and what they're for.	"What command installs package dependencies?"
01:05	25 min	Exercise	Apply macros to the exercise project	Practice	Solo work	Exercise below	Circulate. Task 1 is straightforward. Task 2 is the integration challenge — using the macro in an existing model requires understanding how it compiles. Task 3 introduces dbt_utils. Bonus requires judgment, not code.	All three tasks pass `dbt build`
01:30	10 min	Debrief + prep questions for Module 10	Consolidate; seed next module	Debrief	Verbal	Whiteboard	Debrief on the bonus: was anything in the Silver models a good macro candidate? Drive towards the answer: `generate_surrogate_key` is the right abstraction; most other Silver logic is business-rule SQL and should stay in SQL.	—

Content

Part A — Jinja Syntax: The Three Delimiters

This is a review from Module 03, now applied to macro authoring.

Delimiter	Purpose	Renders output?	Example in dbt context
`{{ }}`	Expression — evaluates and outputs a value	Yes	`{{ ref('fct_deal') }}` · `{{ my_macro(arg) }}`

Delimiter	Purpose	Renders output?	Example in dbt context
`{% %}`	Statement — logic, control flow, definitions	No	`{% if is_incremental() %}` · `{% macro safe_cast(col, type) %}`
`{# #}`	Comment — ignored entirely by Jinja	No	`{# TODO: add null guard here #}`

In macro authoring, you use all three:

```
{# This macro safely casts a column to a target type. #}
{% macro safe_cast(column_name, target_type, fallback=None) %}
    TRY_CAST({{ column_name }} AS {{ target_type }})
{% endmacro %}
```

The `{% macro %}` / `{% endmacro %}` tags are statements — they define, but produce no output. The `{{ column_name }}` and `{{ target_type }}` inside are expressions — they output the argument values into the SQL.

The most common macro authoring mistake:

```
-- ❌ WRONG — uses expression delimiter to define a macro
{{ macro safe_cast(column_name, target_type) }}
    TRY_CAST({{ column_name }} AS {{ target_type }})
{{ endmacro }}

-- ✅ CORRECT — uses statement delimiter
{% macro safe_cast(column_name, target_type) %}
    TRY_CAST({{ column_name }} AS {{ target_type }})
{% endmacro %}
```

The wrong version causes a Jinja parse error immediately. dbt cannot compile any model until it's fixed.

Part B — Built-in dbt Variables

These three variables are available in every model and macro. You've seen `{{ this }}` before (Module 03, Module 04). Here they are together with what they compile to.

`{{ this }}` — the current model's relation

```
-- In a model file, {{ this }} refers to the table this model materialises to.
-- Most common use: incremental watermark filter.

{% if is_incremental() %}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}

-- Compiles to (dev environment):
-- WHERE updated_at > (SELECT MAX(updated_at) FROM dev_schema.fct_deal)
```

`{{ this }}` has three properties: `{{ this.database }}`, `{{ this.schema }}`, `{{ this.identifier }}`. Use them when you need only part of the fully qualified name.

`{{ target }}` — the active profile target

```
-- target.name is the most useful property.
-- Use it for environment-conditional logic.

{% if target.name == 'prod' %}
    -- no row filter — full dataset
{% else %}
    WHERE created_at >= CURRENT_DATE - INTERVAL '90 days'
{% endif %}

-- Compiles to nothing (prod) or a WHERE clause (dev).
```

Common `target` properties:

Property	Value (example)	Use case
`target.name`	`'dev'` / `'prod'`	Environment-conditional logic
`target.schema`	`'TESTING__dev_jane'`	Logging, audit tables
`target.type`	`'duckdb'` / `'snowflake'`	Adapter-specific SQL

`{{ model }}` — the current model's metadata

```
-- model.name is the file name without .sql
-- Less commonly used; mostly useful in generic macros that need to know which model called them.

{{ model.name }}     -- compiles to: fct_deal
{{ model.unique_id}} -- compiles to: model.analytics.fct_deal
```

Part C — Writing Macros

A macro is a reusable Jinja template that takes parameters and produces SQL text. Every macro follows the same anatomy:

```
{% macro macro_name(parameter_1, parameter_2='default') %}
    -- SQL or Jinja that uses the parameters
    {{ parameter_1 }} some SQL {{ parameter_2 }}
{% endmacro %}
```

- The `{% macro %}` tag defines the name and parameters.
- Parameters can have default values.
- Whatever text is between `{% macro %}` and `{% endmacro %}` is what the macro outputs when called.
- Call it with `{{ macro_name(arg1, arg2) }}` from any model or another macro.

Macros are compile-time text substitution. When dbt encounters `{{ safe_cast('amount', 'DOUBLE') }}`, it reads the macro definition, substitutes the arguments into the template, and outputs the resulting SQL. The macro itself never appears in the compiled output — only the result does.

The `safe_cast` macro

Your project uses a `safe_cast` macro so that type coercion failures don't crash models — they silently produce `NULL` instead, which is usually preferable for Bronze data of variable quality.

Definition (`macros/safe_cast.sql`):

```
{# safe_cast: wraps TRY_CAST to produce NULL instead of an error on type mismatch.
   column_name : the column expression to cast (string)
   target_type : the SQL type to cast to (string)
   fallback    : optional default value if cast fails (default: NULL)
#}
{% macro safe_cast(column_name, target_type, fallback=None) %}
    {%- if fallback is not none -%}
        COALESCE(TRY_CAST({{ column_name }} AS {{ target_type }}), {{ fallback }})
    {%- else -%}
        TRY_CAST({{ column_name }} AS {{ target_type }})
    {%- endif -%}
{% endmacro %}
```

The `-` inside `{%- -%}` strips surrounding whitespace from the compiled output. Macro definitions are indented for readability, which would otherwise create blank lines in the compiled SQL.

Calling it in a model:

```
-- models/staging/hubspot/stg_hubspot__deals.sql

SELECT
    deal_id,
    deal_name,
    pipeline_id,
    {{ safe_cast('amount', 'DOUBLE') }}          AS amount,
    {{ safe_cast('close_date', 'DATE') }}        AS expected_close_date,
    {{ safe_cast('deal_score', 'INTEGER', '0') }} AS deal_score,
    _ingested_at AS ingested_at
FROM {{ source('hubspot', 'deals') }}
```

What it compiles to (`target/compiled/...`):

```
SELECT
    deal_id,
    deal_name,
    pipeline_id,
    TRY_CAST(amount AS DOUBLE)                       AS amount,
    TRY_CAST(close_date AS DATE)                     AS expected_close_date,
    COALESCE(TRY_CAST(deal_score AS INTEGER), 0)     AS deal_score,
    _ingested_at AS ingested_at
FROM main.raw_deals
```

No Jinja in the compiled output. Snowflake (or DuckDB) receives plain SQL.

`dbt_utils.generate_surrogate_key`

This is the most-used macro in the project. Every Silver model that needs a surrogate key calls it.

dbt_utils is a collection of pre-written macros. `generate_surrogate_key` IS a macro — just one that the dbt community wrote, not you. You call it the same way you'd call your own macros: `{{ dbt_utils.generate_surrogate_key([...]) }}`.

What it does: hashes one or more columns to produce a consistent, deterministic surrogate key. It uses the adapter's native hash function (MD5 in DuckDB and Snowflake).

Where to call it: in the source CTE of the model, not in a shared macro. Surrogate key generation is structural — it belongs with the row's identity, not in a shared abstraction layer.

```
-- models/silver/fct_deal.sql

WITH source AS (
    SELECT
        {{ dbt_utils.generate_surrogate_key(['deal_id']) }} AS deal_key,
        deal_id,
        deal_name,
        pipeline_id,
        amount,
        expected_close_date,
        ingested_at
    FROM {{ ref('stg_hubspot__deals') }}
)

SELECT * FROM source
```

What it compiles to (DuckDB):

```
WITH source AS (
    SELECT
        MD5(CAST(deal_id AS TEXT)) AS deal_key,
        deal_id,
        deal_name,
        pipeline_id,
        amount,
        expected_close_date,
        ingested_at
    FROM dev_schema.stg_hubspot__deals
)

SELECT * FROM source
```

For multi-column keys, pass all columns that together uniquely identify the row:

```
{{ dbt_utils.generate_surrogate_key(['prescription_id', 'doctor_id']) }} AS prescription_key
```

Part D — When to Use Macros

This is where most teams go wrong. Macros feel powerful, so engineers reach for them to DRY up any repeated code. That's the wrong instinct when the repeated code is business logic.

The rule: use macros for repeated **structural patterns** across models. Avoid macros for **business logic**.

Decision table

Situation	Right tool	Reason
Same CAST expression in 8 staging models	Macro	Structural — no business meaning, purely mechanical
Surrogate key hash over a single column	`dbt_utils.generate_surrogate_key`	Already provided by the package; no need to reinvent
A CASE WHEN that maps pipeline stages to deal categories	SQL CTE	Business logic — needs to be visible, reviewable, testable in isolation
An intermediate join used by exactly one downstream model	SQL CTE or ephemeral model	Single use — macro abstraction adds complexity without benefit
A CASE WHEN that classifies deal size across 15 models	Macro or ephemeral	Repeated business logic — acceptable macro use, but document the business rule in the macro's docstring
A 50-line Jinja template that encodes revenue allocation rules	Never a macro	Jinja is not a good language for business logic — no SQL lineage, no easy test targeting, hard to debug

The antipattern to avoid

```
{# ❌ BAD: business logic encoded in Jinja — hard to test, hard to read #}
{% macro classify_deal(amount_col, stage_col) %}
    CASE
        WHEN {{ stage_col }} = 'closed-won' AND {{ amount_col }} > 50000 THEN 'enterprise-closed'
        WHEN {{ stage_col }} = 'closed-won' AND {{ amount_col }} > 10000 THEN 'mid-market-closed'
        WHEN {{ stage_col }} IN ('proposal', 'demo') AND {{ amount_col }} > 50000 THEN 'enterprise-pipeline'
        -- ... 20 more lines
    END
{% endmacro %}
```

This breaks two things:
1. **Testability** — you can't write a dbt test that targets this macro's output in isolation.
2. **Readability** — the business rule is invisible to SQL analysts and BI developers. They have to know the macro exists and then read Jinja to understand what the data means.

The same logic as a CTE in `fct_deal.sql` is visible in lineage, reviewable in a PR, and testable with an `accepted_values` test.

Code smell checklist

Ask yourself these before writing a new macro:

- Does this pattern appear in more than one model? (If no: use a CTE.)
- Is it purely structural (type coercion, key generation, naming convention) rather than business logic? (If no: use a CTE.)
- Can another analyst understand what it does by reading the calling code without opening the macro file? (If no: reconsider.)
- Would a data analyst reviewing a PR understand the compiled SQL without knowing the macro? (If no: the macro is hurting discoverability.)

Part E — Hooks and Packages

Hooks

Hooks run SQL before or after a model executes. You configure them in `dbt_project.yml` or in a model's `config()` block.

```
# dbt_project.yml — apply a post-hook to all Silver fact models
models:
  analytics:
    silver:
      facts:
        +post-hook:
          - "ALTER TABLE {{ this }} ADD PRIMARY KEY ({{ this.identifier }}_key) NOT ENFORCED"
```

Or inline in a model:

```
{{ config(
    materialized = 'table',
    post_hook    = "COMMENT ON TABLE {{ this }} IS 'Loaded at {{ run_started_at }}'"
) }}
```

Common uses:

Hook	Use case
`pre-hook`	Drop temp tables, set session parameters, audit log start
`post-hook`	Add constraints (DDL), grant permissions, audit log end

Hooks are powerful but hard to debug. Prefer solving problems in SQL or dbt configuration before reaching for hooks.

Packages

Packages are collections of macros, models, and tests you install via `packages.yml`. The two most common in this project:

```
# packages.yml (project root)
packages:
  - package: dbt-labs/dbt_utils
    version: 1.3.0
  - package: calogica/dbt_expectations
    version: 0.10.4
```

Install with:

```
dbt deps
```

This downloads packages to the `dbt_packages/` directory (gitignored). Run `dbt deps` after cloning the repo and any time you add or update a package.

`dbt_utils` macros you will use:

Macro	Purpose
`dbt_utils.generate_surrogate_key([cols])`	MD5 hash surrogate key from one or more columns
`dbt_utils.star(from=ref('model'), except=['col1'])`	SELECT * except listed columns
`dbt_utils.pivot(col, values, ...)`	Pivot a column into multiple boolean columns

`dbt_expectations` tests you will use:

Test	Purpose
`dbt_expectations.expect_column_values_to_be_between`	Range check on numeric columns
`dbt_expectations.expect_column_pair_values_to_be_equal`	Assert two columns match

Live Demo Script

Goal: Write the `safe_cast` macro from scratch, verify it compiles correctly, and use it in `stg_hubspot__deals.sql`.

Time: 15 min

Steps:

1. Open the project. Show that `macros/` directory is empty (or doesn't exist).
2. Create `macros/safe_cast.sql`:
   `sql {% macro safe_cast(column_name, target_type, fallback=None) %} {%- if fallback is not none -%} COALESCE(TRY_CAST({{ column_name }} AS {{ target_type }}), {{ fallback }}) {%- else -%} TRY_CAST({{ column_name }} AS {{ target_type }}) {%- endif -%} {% endmacro %}`
3. Save the file. Run:
   `bash dbt compile --select stg_hubspot__deals`
   Show that the compile fails — the model doesn't call the macro yet.
4. Open `stg_hubspot__deals.sql`. Replace the raw `amount` column with:
   `sql {{ safe_cast('amount', 'DOUBLE') }} AS amount,`
5. Run `dbt compile --select stg_hubspot__deals` again. Open `target/compiled/.../stg_hubspot__deals.sql`.
   Show that `{{ safe_cast(...) }}` has been replaced with `TRY_CAST(amount AS DOUBLE)`.
6. Run `dbt build --select stg_hubspot__deals`. Show green output.

What to highlight: The macro itself never appears in the compiled SQL. dbt is a transpiler — Jinja is an authoring convenience, not a runtime feature. Snowflake and DuckDB know nothing about macros.

Exercise (25 min)

> **Project context:** The exercise project has a staging layer and Silver models from earlier modules. You are adding a shared utility macro and applying it in two places.

Task 1 — Write `macros/safe_cast.sql`

Create the file `macros/safe_cast.sql`. The macro must:
- Take three parameters: `column_name`, `target_type`, and `fallback` (optional, default `None`).
- When `fallback` is provided: output `COALESCE(TRY_CAST(column_name AS target_type), fallback)`.
- When `fallback` is omitted: output `TRY_CAST(column_name AS target_type)`.

Verify it works:

```
dbt compile --select stg_hubspot__deals
```

Check the compiled output in `target/compiled/` to confirm the macro resolves correctly.

▶ Starter skeleton
▶ Expected Result

Task 2 — Use `safe_cast` in `stg_hubspot__deals.sql`

Before starting Task 2, confirm the exercise project has the expected models by running `dbt ls --select stg_hubspot__deals`. You should see one result.

Open `models/staging/hubspot/stg_hubspot__deals.sql`. The `amount` column is currently selected as a raw value. Replace it with a safe cast to `DOUBLE`:

```
{{ safe_cast('amount', 'DOUBLE') }} AS amount,
```

Run:

```
dbt build --select stg_hubspot__deals
```

Open `target/compiled/.../stg_hubspot__deals.sql`. Confirm `amount` now compiles to `TRY_CAST(amount AS DOUBLE)`.

▶ Expected model (amount line only)

Task 3 — Add `dbt_utils.generate_surrogate_key` to `fct_deal.sql`

Before starting Task 3, confirm the exercise project has the expected models by running `dbt ls --select fct_deal`. You should see one result.

Open `models/silver/fct_deal.sql`. Add a surrogate key column `deal_key` by hashing `deal_id` using `dbt_utils.generate_surrogate_key`. Place the key generation in the source CTE, as the first selected column.

```
{{ dbt_utils.generate_surrogate_key(['deal_id']) }} AS deal_key,
```

Run:

```
dbt build --select fct_deal
```

Confirm the model builds successfully. Open the compiled SQL and verify `deal_key` compiles to an MD5 expression.

▶ Expected source CTE (key lines)

Bonus — Macro candidate review

Open any two Silver models (`dim_contact.sql` , `fct_prescription.sql` , or `mrt_deals_funnel.sql`). Find one CTE or expression that repeats across models and answer:

1. Is it a good candidate for extraction as a macro? Why or why not?
2. If yes: write the macro signature (name, parameters). You don't need to implement it.
3. If no: explain what makes it better left as SQL.

There's no single right answer here. The goal is to apply the decision framework from Part D — not to find a predetermined "correct" candidate.

Reference Material

- dbt Jinja functions reference
- dbt macros docs
- dbt hooks docs
- dbt packages docs
- dbt_utils source (GitHub)
- dbt_expectations source (GitHub)
- Internal: `dbt-sql-reviewer` skill — checks for macro overuse and business logic in Jinja pre-merge

---

Prep Questions for Module 10

1. What is a slowly changing dimension (SCD)? What type does dbt's `snapshot` feature implement by default?
2. If a contact's email address changes in HubSpot, how would you know both the old and new email were valid at different points in time?
3. What columns does a dbt snapshot add to a dimension table to track row history?
4. In `dim_contact` , `contact_key` is a hash of `hubspot_contact_id` . If a contact's email changes, does `contact_key` change? Why does that matter for snapshot strategy?